Parallel Computing

A/Prof. Dr. Tran Van Hoai, Dr. Diep Thanh Dang

Assignment 1

(September 18, 2026)

1 Mutex Locks

In the lecture, we discussed several spinlock algorithms (e.g., TAS/ TTAS/ MCS). In this assignment, we are about to do the following:

- Implementing TAS/ TTAS/ MCS for multi-threaded programs using C++.
- Devising the approach to testing the correctness of your implementations.
- Testing several variants of the mutex locks in a multi-threaded program consisting of the following C++ code fragment:

```
#include <chrono>
#include <thread>
#include "lock.h"

using namespace std::chrono;

/* Configurations */
#define NUM_OF_THREADS  ...
#define NUM_OF_ITERS  ...
#define OUTSIDE_WORK  ...
#define N   ...

/* Shared variables */
lock_mcs  lock; // for example: MCS Lock

/* Private variables */
// thread_local int dummy;

/* Child thread's code */
void thread_entry(...)
{
        for (auto i = 0; i < NUM_OF_ITERS; ++i)
  {
    lock.acquire();

    // check if a number N is a prime
    is_prime(N);

    lock.release();

    for (auto j = 0; j < OUTSIDE_WORK; ++j)
      is_prime(N);
  }
}
```

is_prime() is meant to represent work done inside the critical section and implements a simple primality test.

```
int is_prime(int p)
{
  int d;
  for (d = 2; d < p; d = d + 1)
    if (p % d == 0)
      return 0;
  return 1;
}
```

The amount of contention is determined by the value of OUTSIDE_WORK. A value of 0 means that all work is done in critical sections, 1 means that half of the work is done in critical sections, 2 means that 1/3 of the work is done in critical sections, and so on.

- Conducting a performance and scalability study of all the mutex lock algorithms on all shared-memory platforms (e.g., your notebooks) you possess by determining execution time for the whole program as you tune predefined parameters. Comparing the performance of your implementation to any existing mutex lock(s) you can find (e.g., OpenMP, Pthreads, C++, etc.).
  Note that the exponential backoff can be used in some mutex lock algorithms to minimize contention between threads. The backoff refers to a waiting time before retrying a failed increment attempt that can be used with the compare-and-swap solutions. The exponential backoff can be achieved in C++ like so:

```
#include <chrono>
#include <thread>

#define MAX_DELAY ...

unsigned delay = ...;
while (!compare_exchange(...))
{
  std::this_thread::sleep_for(std::chrono::nanoseconds(delay));
  if (delay < MAX_DELAY) {
    delay *= 2;
  }
}
```

2 Barrier Synchronization

In the lecture, we discussed how Sense-Reversing Barrier works. The goal of this assignment is to:

- Implement Sense-Reversing Barrier for multi-threaded programs in C++.
- Devise the approach to testing the correctness of your implementation.

- Conduct a performance and scalability study of your barrier on all shared-memory platforms you possess. Considering the number of completed barrier episodes per unit time with no useful work between barriers as a performance metric. Comparing the performance of your implementation to any existing barriers(s) you can find (e.g., OpenMP, Pthreads, C++, etc.). Note that the exponential backoff can be used in some barrier synchronization algorithms to minimize contention between threads.

3 Concurrent Counters

In the lecture, we discussed several implementations of concurrent counters using various kinds of synchronization (e.g., mutex locks, compare-and-swap, fetch-and-add). This assignment aims to:

- Implement concurrent counters using all kinds of synchronization mentioned above using C++.
- Devise the approach to testing the correctness of the concurrent counter implementations.
- Conduct a performance and scalability study of all the concurrent counter versions on all shared-memory platforms you possess. Considering the execution time as a performance metric. Note that the exponential backoff can be used in some concurrent counter implementations to minimize contention between threads.

4 Distributed Versions (optional)

All the aforementioned data structures and algorithms can only be run on shared-memory platforms, not distributed-memory ones (e.g., supercomputers). Distributed-memory platforms are more scalable than shared-memory counterparts in terms of hardware. In this assignment, you are supposed to implement their distributed versions using MPI-3 One-Sided Communications (see Chapter 12 in Message Passing Interface Standard specification version 4.1[1] for further details). MPI-3 One-Sided Communications support various distributed-memory programming primitives analogous to those of C++11 Concurrency. Hence, it is possible to port all the shared-memory data structures and algorithms you implemented so far into distributed-memory environments using MPI-3 One-Sided Communications.

Other Requirements

This assignment must be conducted in groups, each of which consists of either 2 or 3 members. You must submit a zip file containing your final pdf report as well as all

[1] https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf

relevant C++ source code to our course website by October 31, 2026. Especially, the number of experimented shared-memory platforms must at least be equal to the member count in your group.